HaloPoint 3.0

Jukka Ruoskanen[1]*

Abstract
HaloPoint 2.0 came out a long time ago. The halo dots appeared lazily on the screen, and right from the start, it was clear that a speed boost would make a huge difference. I'd always had that on my to-do list, but that list is long—full of all kinds of interesting things one wants to get done in life. I'm sure that, in many people's minds, any hope for a new version of HaloPoint had long since been buried. But here it is—finally! This spring, I decided it was time to elevate the software into the modern age: GPU acceleration, non-convex crystal handling, and a GUI that's hopefully more user friendly than the previous version. Over the years, I'd played around with parallelization and other algorithmic optimization strategies, but I never managed to finish them. This time, the momentum was there. ❄ This brief manual explains the basics of the software. Users are encouraged to report all bugs to the email address below. And those who are interested in more detailed explanation of what happens under the hood, feel free to be in contact. ❄ Enjoy simulation experience that is 1000 times faster than before!

[1] *HaloPoint 3.0, (2026), https://unzi-24.github.io/halopoint3/*
*__Correspondence__: halopoint.3@outlook.com

Contents

Introduction 1
1 Quick Start 3
1.1 Overview of the main window . . . 3
1.2 Setting up a basic simulation . . . 3
1.3 Setup files and session persistence . . . 3
2 Simulation Parameters 4
2.1 General parameters . . . 4
2.2 Simulation view . . . 5
2.3 Camera projection . . . 5
2.3.1 Custom lens distortion . . . 5
2.4 Simulation appearance . . . 5
2.4.1 Halo ring overlays . . . 6
2.5 GPU acceleration . . . 6
2.6 Why only CUDA and CPU? . . . 7
2.7 Multiple scattering . . . 7
2.8 The simulation window . . . 8
2.9 Recording mode . . . 9
3 Crystal Populations 10
3.1 Parametric crystals – dimensions . . . 10
3.2 Crystal orientation . . . 11
3.3 OBJ file crystals (convex and non-convex) 12
3.4 Managing multiple populations . . . 13
3.5 Crystal preview . . . 13
4 Advanced Functions 14
4.1 Raypath filtering . . . 14
4.2 Batch processing . . . 15
5 Analysis Window 16
5.1 Opening the analysis window . . . 16
5.2 Raypath histogram and bins . . . 16
5.3 Raypath illustration . . . 18
Appendix — Simulation path capabilities 19
Suggestions for further reading 19

Introduction

HaloPoint 3.0 is a simulation tool for atmospheric ice crystal halos — a richly varied family of optical phenomena visible in the sky when sunlight interacts with tiny airborne ice prisms. Dozens of distinct halo forms have been documented, and new ones continue to be discovered. Their variety stems from the many shapes and orientations that ice crystals can take, each allowing light to travel a different path: from simple refraction through a prism wedge, to complex routes involving several internal reflections in succession. HaloPoint 3.0 traces these paths computationally and projects the resulting halo pattern onto the celestial sphere, producing a simulation image that can be directly compared with a photograph taken in the field.

The software is designed for anyone with an interest in atmospheric optics — from the hobbyist who wants to identify an unusual halo in a snapshot, to the researcher examining the geometry of exotic ray paths. No programming knowledge is required. The interface is organized around a small number of clearly defined parameters: the shape and orientation of the ice crystals, the position of the sun, and the characteristics of the camera used to photograph the sky. Adjusting these parameters and running the simulation typically takes just a few minutes, and the results update immediately in a dedicated simulation window.

System requirements and installation. HaloPoint 3.0 runs on Microsoft Windows and requires the .NET 10 Desktop Runtime. If it is not already installed on

Key Features

★ Performance

GPU-accelerated ray tracing processes millions of rays per second. Multiple-scattering phenomena are fully supported.

★ Batch Processing

Queue a series of simulation runs and let the software execute them unattended, sweeping over any parameter range automatically.

★ Crystal Shapes

Standard hexagonal crystals and arbitrary user-defined non-convex shapes can be used in both single- and multiple-scattering simulations. Crystal dimensions are sampled from probability distributions according to the users instructions.

★ Simulation Appearance

Simulate across the full colour spectrum, at selected wavelengths, or in monochrome. Shade levels are configurable from 1 to 1 million and the outcome can be exported as a 32 bit float TIFF that preserves the full dynamic range.

★ Ray Path Analysis

A rich set of filtering tools lets you isolate and examine specific halo forms. Each dot in the simulation image can be traced back to its exact ray path through the crystal. For further analysis the raypath histogram can be exported as csv or json file.

★ Camera Matching

Match the simulation to a photograph by specifying focal length, sensor size, and image centre point in the sky. Custom radial distortion coefficients are supported for precise alignment.

★ Crystal Visualization

Crystal shapes and orientations are rendered interactively. A ray path illustration shows the exact trajectory of light through the crystal for any selected simulation point, including multiple scattering.

★ Parameter Distributions

Every variable governing crystal dimensions, shape, and orientation can be set to vary according to a uniform or normal distribution, with a user-defined interval or standard deviation.

your computer, Windows should prompt you automatically. If not, download and install the .NET 10 Desktop Runtime from Microsoft and then launch HaloPoint again.

The software runs on any modern Windows PC. A dedicated GPU is not required. On computers without one, all simulations run on the CPU, which is fully capable of producing results, though somewhat more slowly than GPU-accelerated mode. For the best performance, a CUDA-capable NVIDIA GPU is recommended; OpenCL is not supported (see subsection “Why CUDA and GPU?”). When a compatible GPU is detected at startup, HaloPoint 3.0 automatically compiles its ray tracing kernels to run on the hardware; this compilation happens once in the background and may take between a few seconds and roughly a minute, depending on the GPU and the simulation mode selected. A status indicator in the main window shows when the GPU is ready.

To install the software, extract the downloaded archive to a folder of your choice and run HaloPoint3.exe. No installer is required. Session state is saved automatically to your user profile (`%LocalAppData%\HaloPoint3`), but no files are written elsewhere outside the application folder, making it straightforward to move or remove the software.To install the software, extract the downloaded archive to a folder of your choice and run HaloPoint3.exe.

About this software. HaloPoint 3.0 is a free software, and intended for personal use. You may not redistribute or sell this software in websites or by any other means. HaloPoint 3.0 is available for download in the hope that it will be useful, but it comes without any warranty. Extensive testing and validating measures have been taken to ensure correctness of the simulations, but since the code has been written as a hobby and not by a professional software developer, there may lurk some yet unidentified errors within the software. Users are encouraged to report all software faults and exceptions to the email address which can be found at the software homepage (currently: halopoint.3@outlook.com). Please, acknowledge the use of HaloPoint 3.0 in publications and websites and use the following reference in your publications:

> HaloPoint 3.0
> https://unzi-24.github.io/halopoint3/

Third-party notices. HaloPoint 3.0 makes use of the following third-party libraries. **ILGPU** (copy-

right © 2016–2025 ILGPU Project) is used for GPU-accelerated ray tracing and is licensed under the University of Illinois/NCSA Open Source License—a permissive licence whose conditions require the preservation of copyright and licence notices in source and binary distributions. **Xceed Extended WPF Toolkit™** (copyright © Xceed Software, Inc.) provides certain user-interface controls and is used under the Xceed Community Licence for non-commercial use. Full licence texts for both libraries are reproduced in the *About* dialog, accessible from the toolbar of the main window (**i About** → *Third-Party Notices*).

1. Quick Start

1.1 Overview of the main window

When HaloPoint 3.0 opens, you are presented with the main window — the central hub for configuring and launching simulations. The simulation itself runs in a separate window that appears when you click RUN SIMULATION, so the main window stays open throughout a session.

The main window is divided into three visual areas: a top bar, a tab area that occupies the bulk of the window, and a status bar along the bottom.

The top bar. The top bar contains four buttons. LOAD SETUP and SAVE SETUP open and write configuration files. RUN SIMULATION launches the simulation; GPU-accelerated runs are unavailable until kernel compilation completes at startup, but CPU-mode simulations can be started immediately. Once compiled, the GPU kernels remain available for the lifetime of the session and do not need to be recompiled on subsequent simulation runs. The ABOUT button opens the About dialog, which contains info about HaloPoint 3.0 and the full third-party license texts.

The tab area. The main content is organized across five tabs:

- SIMULATION PARAMETERS — sun position, number of rays, wavelength range, and camera projection.
- CRYSTAL POPULATIONS — up to ten independent crystal populations, each with its own shape, dimensions, and orientation distribution.
- ADVANCED FUNCTIONS — multiple scattering, recording mode, custom camera lens parameters and GPU tuning.
- RAYPATH FILTERING — rules for highlighting or isolating specific halo forms based on the ray paths through the crystals.
- BATCH PROCESSING — a queue of simulation runs that execute sequentially without further interaction.

The notification bar and status bar. A row of small indicator badges sits to the right of the tab labels. Cyan badges indicate active features such as GPU acceleration or multiple scattering; amber badges flag conditions that require attention, for example when a settings combination causes the simulation to fall back from GPU to CPU. The status bar at the bottom reports the state of the GPU — whether kernels are still compiling, fully ready, or unavailable.

1.2 Setting up a basic simulation

A simulation requires three things to be configured: the physical conditions in the SIMULATION PARAMETERS tab, the crystal populations in the CRYSTAL POPULATIONS tab, and the camera projection, also in SIMULATION PARAMETERS.

Sun and rays. Set the sun elevation to match your scenario. Ray count and shade levels together determine image appearence: more rays produce a smoother result, but also requires a greater dynamic range, which can be achieved by increasing the number of shade levels.

Crystal populations. Switching to the CRYSTAL POPULATIONS tab a list of ten populations on the left side of the tab can be seen; the currently active populations are indicated by a green dot. For each active population a crystal shape, dimensions, and orientation with related distributions are defined. The orientation preset dropdown provides sensible initial values for the most common crystal orientations. Up to ten populations can be active simultaneously.

Camera projection. The image center settings define where the image is centered in the sky and the camera projection dictates field of view, controlled by the optical lens focal length and sensor size.

Running. Clicking RUN SIMULATION opens the simulation window where halo dots begin to accumulate immediately, with the image building up progressively as more rays are traced. The bottom strip of the simulation window shows the number of rays traced so far out of the total, the number of halo points produced, elapsed time, and the current tracing rate in rays per second. When you are satisfied with the image, click STOP to halt the simulation, or let it run to completion. Clicking SAVE IMAGE opens a file dialog where you can export the result as an 8, 16 or a 32 bit image file. Multiple simulation windows can be open at the same time; you can adjust parameters and launch additional simulation windows if you wish to compare results side by side.

1.3 Setup files and session persistence

Saving and loading. Click SAVE SETUP or LOAD SETUP in the top bar to write or restore a setup file. A setup file captures the complete state of the software. Setup files use the `.hpsetup` extension and are stored as plain text, so they can be opened in a text

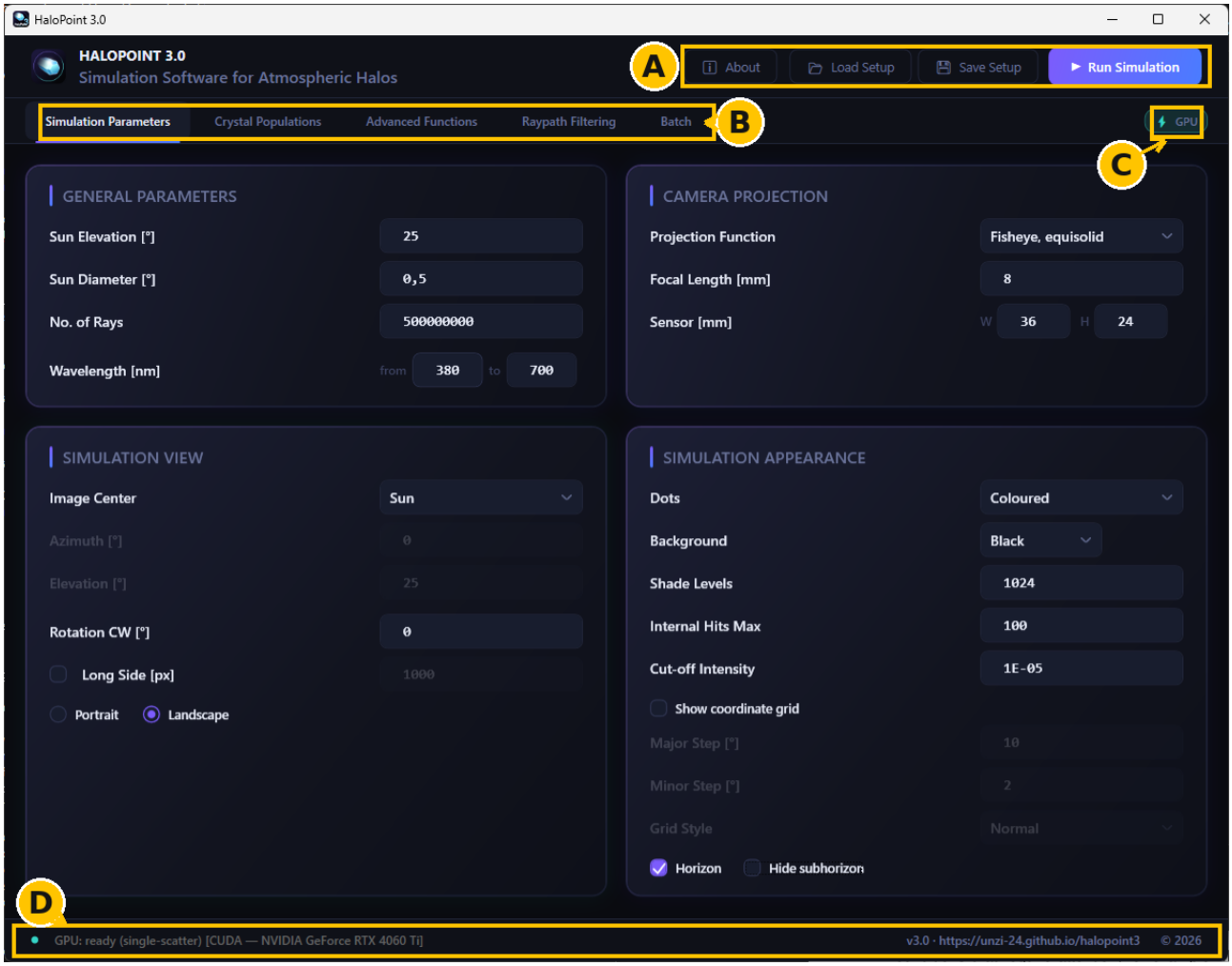


Figure 1. *The HaloPoint 3.0 main window. The top bar (**A**) holds the About, Load Setup, Save Setup, and Run Simulation buttons. The five tabs (**B**) organise all simulation settings. The notification badges (**C**) report active features and warnings, and the status bar (**D**) shows kernel compilation progress and general state.*

editor and shared between users directly. When loading, all current settings are replaced immediately and any unsaved changes are discarded without a warning prompt. It is good practice to save before making significant changes to a working configuration, particularly when experimenting with crystal parameters to match a display.

Saving the simulation image. Click SAVE IMAGE in the simulation window at any point during or after a run to export the current image as BMP or an 8 or 16 bit PNG or TIFF or a 32-bit float TIFF that stores the raw accumulation buffer itself for quantitative analysis. The default filename includes a timestamp in the format `halo_YYYYMMDD_HHMMSS`.

Autosave and session restore. HaloPoint 3.0 saves your current configuration automatically each time the application closes. When you reopen the software, it restores this autosave silently, so you continue from where you left off without any manual action. The autosave is stored in your user profile separately from any explicitly saved setup files and cannot be accessed through the LOAD SETUP dialog.

2. Simulation Parameters

2.1 General parameters

The GENERAL PARAMETERS card is in the upper-left of the Simulation Parameters tab. It contains the four fundamental inputs that define the physical conditions of every simulation:

SUN ELEVATION sets the elevation of the sun above the horizon in degrees. Valid values range from −90 to 90 degrees.

SUN DIAMETER sets the angular size of the light source. The physical value of the Sun and the Moon is approximately 0.5 degrees, which is the default.

NO. OF RAYS sets the total number of light rays the simulation traces. Notice, that this number is not the number of halo points created in the sky. If a ray misses a crystal, or does not get plotted for other reasons, the ray counter is still incremented. Therefore the resulting number of halo points in any simulation is going to be less than the number entered in this textbox.

WAVELENGTH accepts two values defining the range of light wavelengths sampled, from a minimum

to a maximum, both in nanometers. The allowed range is 380 to 700 nm.

2.2 Simulation view

The SIMULATION VIEW card is in the lower-left of the Simulation Parameters tab. It controls where the simulation image is pointed in the sky, how the camera is oriented, and the pixel dimensions of the output.

IMAGE CENTER defines the celestial point placed at the center of the simulation image. In the dropdown there are eight special celestial points available. The ninth entry in the list is *User Defined*, which when selected activates the AZIMUTH and ELEVATION fields directly below, which accept values in the range -180 to 180 degrees and -90 to 90 degrees respectively, letting you point the image at any direction in the sky. These two fields are greyed out and inactive for all other image center options.

ROTATION CW rotates the camera clockwise around its viewing axis, in degrees from -180 to 180.

LONG SIDE sets the length of the longer dimension of the output image in pixels. The valid range is 100 to 10,000 pixels. This field is enabled by a checkbox directly to its left; when the checkbox is unchecked, the output image is sized automatically based on the size of the user's screen. This feature is especially useful if the user is looking to export a high resolution simulation image.

The PORTRAIT and LANDSCAPE radio buttons at the bottom of the card control the image orientation. Landscape is selected by default, which places the sensor width along the horizontal axis. Portrait swaps this, making the image taller than it is wide.

2.3 Camera projection

The CAMERA PROJECTION card is in the upper-right of the Simulation Parameters tab. It defines the optical properties of the simulated camera — that is, how the sky is projected onto the flat simulation image. These settings should reflect the actual lens and sensor of the camera used to take a halo photograph when the goal is to match the simulation against it.

PROJECTION FUNCTION is a dropdown selecting the mathematical model used to map angular sky positions to image positions. Six options are available: *Rectilinear*, *Fisheye equisolid*, *Fisheye equidistant*, *Stereographic*, *Orthographic*, and *Samyang*. Of these, the Rectilinear, Fisheye equisolid and Samyang are the most common commercially available lens types. The others are specialized projections which are included just in case someone finds use for them.

FOCAL LENGTH sets the physical focal length of the lens in millimeters, in the range 0.1 to 1000 mm.

SENSOR accepts the width and height of the camera sensor in millimeters, labeled W and H. Together with the focal length, these determine the field of view.

2.3.1 Custom lens distortion

The Advanced Functions tab provides an additional camera modeling option: ENABLE CUSTOM LENS DISTORTION. When this checkbox is checked, the polynomial lens mapping

$$r = f \cdot (\theta + k_1\theta^3 + k_2\theta^5 + k_3\theta^7) \tag{1}$$

replaces the Tab 1 projection function entirely. Here r is the radial distance on the sensor in millimeters, f is the focal length, and θ is the angle from the optical axis in radians. There are several good basis functions for estimating the radial mapping function. Function 1 was chosen for its simplicity and sufficient accuracy. The coefficients k_1, k_2, and k_3 are entered in the three fields below the formula. Negative k_1 produces barrel distortion; positive k_1 produces pincushion distortion.

This mode is intended for photographers who have calibrated their lens using a known calibration target, for example a starfield, and fitted the function (1) to the true radial mapping curve. The calibration can be performed also using dedicated software such as PTGui or Hugin, which produce k-coefficients in this same convention. Entering the calibrated coefficients gives a simulation whose geometry estimates the actual radial optical distortion of the lens being used, if the ideal mapping functions do not give a satisfactory overlay comparison with a photograph. The custom lens feature is off by default and can be ignored for most simulation work.

2.4 Simulation appearance

The SIMULATION APPEARANCE card occupies the lower-right of the Simulation Parameters tab. It governs the visual rendering of the halo image — the color of dots, the background, the brightness dynamic range, and optional reference overlays drawn on top.

DOTS is a dropdown controlling how halo dots are colored. *Coloured* (the default) maps each ray's wavelength to a visible color using a spectral lookup table. *White* and *Black* renders all dots as white or black, regardless of wavelength.

BACKGROUND sets the color of the simulation canvas. The options are *Black*, *White*, and *User Defined*. User Defined reveals a color picker button for selecting any background color. White combined with Black dots or vice versa produces a monochrome result.

SHADE LEVELS controls how many halo dots striking the same pixel are needed to saturate it to full brightness. Each dot contributes 1/ShadeLevels of full intensity to the pixel it lands on. At a value of 1, a single hit saturates the pixel immediately. Higher values preserve dynamic range in dense halo regions — where a bright halo core would otherwise clip to a uniform white — at the cost of requiring more rays before faint features become visible. The valid range is 1 to 1000000 (1M).

INTERNAL HITS MAX sets an upper limit on the number of surface encounters a single ray may undergo inside a crystal before being discarded. This prevents

rare degenerate paths from consuming disproportionate computation. The default of 100 is generous and rarely needs adjustment.

CUT-OFF INTENSITY sets the minimum ray intensity below which a ray is abandoned. At each surface interaction inside a crystal, save for total internal reflection, a ray splits into a reflected and a refracted component; the weaker component's intensity diminishes with each encounter. Once intensity falls below this threshold, the ray is discarded. The default of 0.00001 is conservative: rays cut off at this level carry so little energy that they contribute next to nothing visible to the image.

SHOW COORDINATE GRID is a checkbox that overlays a dot-based angular coordinate grid on the simulation image. When enabled, three further controls become active: MAJOR STEP and MINOR STEP set the angular spacing of the primary and secondary grid lines in degrees, and GRID STYLE offers *Normal*, *Strong*, or *Faint* options that adjust the visual weight of the grid dots.

HORIZON and HIDE SUBHORIZON are two further checkboxes. HORIZON draws a line at 0° elevation across the image. HIDE SUBHORIZON suppresses all halo dots at negative elevation, which is useful when you want to avoid accumulating dots in that region.

2.4.1 Halo ring overlays

The Advanced Functions tab contains a second set of appearance controls that extend the Simulation Appearance card: reference halo ring overlays drawn as concentric circles centered on the sun. Eight standard radii are available as toggleable chip buttons: 9°, 18°, 20°, 22°, 23°, 24°, 35°, and 46°. Toggling a chip draws a ring at that angular radius centered on the sun. Four additional free-entry fields accept custom radii in degrees for any non-standard radius of interest.

A separate PARHELIC CIRCLE checkbox draws a full parhelic circle in the sky.

The DRAWING STYLE radio buttons — *Normal*, *Strong*, and *Faint* — control the visual weight of all ring and parhelic circle overlays.

2.5 GPU acceleration

GPU acceleration is configured in the PROCESSING PARAMETERS card on the Advanced Functions tab. This card combines three groups of controls: the GPU enable toggle and hardware identification, the GPU-specific performance parameters, and the non-convex crystal tracing parameters. The first two groups are covered here; the non-convex parameters are addressed in Chapter 3 alongside the OBJ crystal types they relate to.

Hardware identification and enabling. RECOGNIZED GPU is a read-only label that displays the name and interface of the GPU detected on your system. HaloPoint 3.0 probes for hardware at startup, using a CUDA-capable NVIDIA device if one is present and otherwise falling back to a software CPU path. The label shows a description in the form "CUDA — [device name]" or "CPU (software fallback)" if no CUDA device is available. Note that this means a non-NVIDIA GPU — including Intel and AMD integrated graphics — is reported as "CPU (software fallback)": the GPU is present but is deliberately not used for computation, for the reasons given in the following subsection. This label updates as soon as detection completes.

ENABLE GPU ACCELERATION is a checkbox that turns the GPU engine on or off. It is enabled by default. When unchecked, the simulation falls back entirely to the CPU engine regardless of what hardware is present. This can be useful for troubleshooting or on machines where the GPU is needed for other tasks during a long simulation run. When GPU acceleration is active but the relevant compute kernel has not yet finished compiling, the RUN SIMULATION button remains disabled until compilation completes — the status bar at the bottom of the main window reports the current kernel state.

The GPU notification badge in the tab bar reflects the same state: amber while compiling, cyan when ready, and red if compilation failed.

ABOUT THE KERNEL COMPILE TIMES. On startup, HaloPoint compiles its CUDA kernels in the background, and the compile time depends on which kind of simulation you run. The figures below were measured on an NVIDIA GeForce RTX 4060 Ti and are approximate; exact times vary with driver version and hardware. The basic kernel — the one used for ordinary single-scattering simulations — compiles almost instantly, in around 10 seconds, and is the first to be built when the program starts. The recording version, which additionally captures the ray path data needed for analysis, adds very little to this and compiles in essentially the same time, about 11 seconds. The multiple-scattering kernel is heavier, taking roughly 100 seconds to compile. It is compiled only when a simulation actually requires it, rather than at every startup. (Multiple scattering involving non-convex crystals is handled by the CPU engine rather than a GPU kernel; see below.) Importantly, this compilation happens only once per session. Once a kernel has been built, it stays ready for the rest of the time the program is running, so any subsequent simulation that uses it starts immediately with no further wait. You pay the compile cost the first time you run a given type of simulation, and not again until you close and reopen the software.

GPU parameters. These two controls tune how the GPU engine delivers results to the screen during a running simulation. They are greyed out when GPU acceleration is disabled.

BATCH SIZE is a dropdown controlling how many rays are dispatched to the GPU in each kernel launch. The available options range from 16,384 (16K) to 1,048,576 (1M). The default of 131,072 (128K) is a good balance for most consumer GPUs: large enough to saturate the GPU's parallel execution units, small

enough not to delay the first visible results for too long. Higher batch sizes can improve throughput on high-end GPUs with many cores, while lower values produce more frequent progress updates at the cost of some efficiency. This setting rarely needs adjustment, but is available for performance fine tuning.

SNAPSHOT INTERVAL controls how often the GPU's internal high-dynamic-range accumulation buffer is copied back to the CPU for display, in milliseconds. The default of 30 ms gives approximately 33 display refreshes per second, which is enough to see the image building smoothly. Reducing this value makes updates more frequent but increases the data transfer load between GPU and CPU memory. Increasing it reduces that load on long runs — useful if the PCIe transfer overhead is measurable on very bandwidth-constrained systems. For most users the default requires no change.

2.6 Why only CUDA and CPU?

HaloPoint 3.0 accelerates simulations on NVIDIA GPUs through the CUDA backend, and runs on the CPU everywhere else. It does not use OpenCL, even though the underlying GPU library (ILGPU) is capable of targeting it. This subsection explains the reasoning, since users with an Intel or AMD GPU will reasonably wonder why their hardware is not put to work.

The short version. On an NVIDIA GPU, the CUDA path is fast and produces results identical to the CPU reference engine. On other GPUs the only available route is OpenCL, and the OpenCL implementations shipped with Intel integrated graphics were found to compile the simulation kernels extremely slowly and, worse, to compute them incorrectly — producing halo images that are silently wrong rather than merely slow. Faced with the choice between a fast GPU result that cannot be trusted and a slower CPU result that is always correct, HaloPoint chooses correctness. A non-NVIDIA machine therefore runs on the CPU, which is the same engine used to validate every GPU result.

What goes wrong on OpenCL. The problem is not theoretical; it was observed directly with different OpenCL hardware setups. HaloPoint constructs its parametric crystal geometry inside the GPU kernel at run time, which involves a substantial amount of floating-point and transcendental computation per ray. On the Intel OpenCL compiler this kernel math was evaluated incorrectly: the simulation produced results that were inconsistent and visibly wrong, manifesting plain errors in the geometry the kernel computed, even though the kernel compiled and ran without reporting any fault. This is precisely the kind of failure that is dangerous in a scientific tool — an image that is silently incorrect, which a user who did not already know the expected result would have no way to recognise as wrong. The more complex kernels fared worse still: the multiple-scattering kernel was essentially incapable of putting anything on the screen at all, and monitoring the GPU load during these runs showed behaviour that did not look healthy, indicating that the kernel and the Intel OpenCL implementation were fundamentally not getting along.

Why this is hard to fix in general. In all honesty, the foremost reason is surely that the developer's skill reached its apex ☺. That said, there is a bag of contributing explanations worth setting out. Several backend-specific quirks were identified and individually addressed during investigation, but each fix exposed another, and the deeper issue is structural rather than a single bug. The Intel OpenCL toolchain is no longer actively developed — Intel itself has declared its legacy OpenCL SDK end-of-life and steers developers toward newer frameworks — so its compiler behaviour cannot be relied upon to improve, and the differences between it and the mature CUDA toolchain are not all documented or predictable. Verifying correctness would mean validating the full simulation against every relevant Intel driver version indefinitely, on hardware whose GPU compute performance is in any case modest. The cost of guaranteeing trustworthy OpenCL output is therefore open-ended, while the benefit — a marginal speed-up on integrated graphics that is often no faster than a modern multi-core CPU for this workload — is small. Removing OpenCL entirely is the only rational conclusion: rather than offer a GPU path that is correct on some drivers and silently wrong on others, HaloPoint offers it only where it has been verified.

What this means in practice. If your computer has an NVIDIA GPU, leave GPU acceleration enabled and expect fast, correct results. If it does not, the RECOGNIZED GPU label will read "CPU (software fallback)" and simulations will run on the processor. The CPU engine is slower — substantially so for multiple-scattering or large ray counts — but it is the reference implementation and is correct on every machine. It remains fully featured: every crystal type, halo, and analysis function is available on the CPU exactly as on the GPU.

2.7 Multiple scattering

Multiple scattering is configured in the MULTIPLE SCATTERING card on the Advanced Functions tab. It is off by default and not needed for the majority of simulations.

What multiple scattering does. In a standard simulation, each ray of light passes through exactly one ice crystal before either producing a halo dot or being lost. In some situations a ray can exit one crystal and enter another, undergoing a second round of refraction and reflection. This secondary encounter — and potentially a tertiary one — are known to produce interesting halos. Multiple scattering mode enables these multi-crystal ray paths in the simulation.

Controls. ENABLE MULTIPLE SCATTERING is a checkbox that activates the mode. Ticking it reveals the two

controls below, which are otherwise greyed out.

PROBABILITY is a slider and paired text field, ranging from 0 to 1. It sets the probability that a ray emerging from one crystal is offered to a second crystal rather than producing a halo dot directly. At 1.0, every outgoing ray always attempts a secondary encounter; at 0, multiple scattering is effectively inactive even if the checkbox is ticked. Intermediate values balance the proportion of singly-scattered versus multiply-scattered dots in the image, which in turn affects how bright each contribution appears relative to the other. In physical terms, the appropriate value depends on the crystal number density in the cloud being modeled, which is not directly measurable from a photograph.

HaloPoint 2.0 offered a mode that attempted to model physical conditions more accurately than a single probability value allows. For a given population, the user could define a layer thickness together with a mean-free-path profile within that layer, making it possible to approximate the true mean free path inside a crystal layer by adjusting these parameters until the outcome was satisfactory. User feedback, however, was that this mechanism felt cumbersome, and in practice almost everyone preferred the single probability value instead. HaloPoint 3.0 therefore adopts that simpler approach.

MAX SCATTERING LEVEL offers two options via radio buttons: *Level 2* and *Level 3*. Level 2 (the default) allows rays to pass through at most two crystals — a primary and one secondary. Level 3 allows a further tertiary encounter.

Performance and engine behavior. When multiple scattering is enabled, HaloPoint 3.0 requires a different GPU compute kernel from the standard single-scatter mode. If GPU acceleration is on, this kernel is compiled in the background when multiple scattering is first enabled — the status bar reports its progress, and the RUN SIMULATION button remains active for single-scatter runs while the MS kernel compiles. Once ready, GPU multiple scattering runs efficiently and is the recommended mode for standard use.

Two combinations cause an automatic fallback to the CPU engine, indicated by an amber notification badge in the tab bar. If multiple scattering and recording mode are both enabled simultaneously, the simulation runs on CPU — GPU recording does not support multiple scattering. Likewise, if multiple scattering is combined with non-convex OBJ crystals, the simulation runs on the CPU engine; the GPU offers no advantage for this combination (it proved nearly equal or even a bit slower performance than the CPU engine in testing – again the limits of the developer were reached) and so no GPU kernel is provided for it. These fallbacks produce correct results but are slower than pure GPU single-scatter mode.

2.8 The simulation window

Clicking RUN SIMULATION opens the simulation window, which takes over the screen as a maximized window and displays the halo image as it accumulates in real time. The simulation window is fully independent of the main window — it appears as its own entry in the taskbar and can be moved freely — so both remain accessible simultaneously. Multiple simulation windows can be open at once, each running or displaying a separate result.

Layout. The window has three areas: a top strip with the two action buttons, a scrollable canvas showing the image, and a bottom strip of live statistics.

Top strip — Save Image and Stop. SAVE IMAGE opens a file dialog during or after a simulation. Six formats are offered: PNG and TIFF each in 8-bit and 16-bit variants, a BMP, and a 32-bit float TIFF. The default filename takes the form `halo_YYYYMMDD_HHMMSS`, and the dialog remembers the last format used. PNG 8-bit is recommended for general use; the 16-bit PNG and TIFF options preserve far greater tonal range, rendering the float accumulation buffer directly rather than the 8-bit screen image, and are the better choice when the result will be processed or stretched afterward. The 32-bit float TIFF stores the raw accumulation buffer itself for quantitative analysis (see below). The saved image captures the accumulation as it stands when the dialog is confirmed; saving does not interrupt the run. STOP halts ray tracing immediately and freezes the image, which can still be saved; the button then reads "✓ Stopped". A simulation that finishes its full ray budget on its own instead shows "✓ Done".

The 32-bit float TIFF. All of the other formats store the *tone-mapped* image — the picture as it appears on screen, with the brightest halos clipped to white once a pixel is fully exposed. The 32-bit float TIFF instead stores the simulation's raw accumulation buffer without tone-mapping or clipping, so it is intended for quantitative analysis rather than direct viewing. Each pixel records how much light landed there in normalized units: one ray contributes $1/\text{shade levels}$ to a channel, so a pixel that has received exactly "shade levels" rays reads 1.0 — the level at which the displayed image reaches full white. Pixels brighter than this are not clipped; they simply exceed 1.0. A pixel that received five times that many rays reads 5.0, a value the integer formats cannot represent. To recover the underlying ray count for a pixel, multiply its stored value by the shade-levels setting used for the run. This makes the float TIFF the format to use when the actual brightness ratios between parts of a halo display matter — for instance when comparing the relative intensity of two arcs — since that information survives in the stored numbers even where the visible image is saturated.

Bottom strip — progress statistics. Updated about ten times per second, the strip reports: "Rays" as traced versus requested (n/N); "Halo pts", the number of dots plotted so far (always lower than the ray count, since a ray only yields a dot when it exits a crystal within the camera's field of view); the

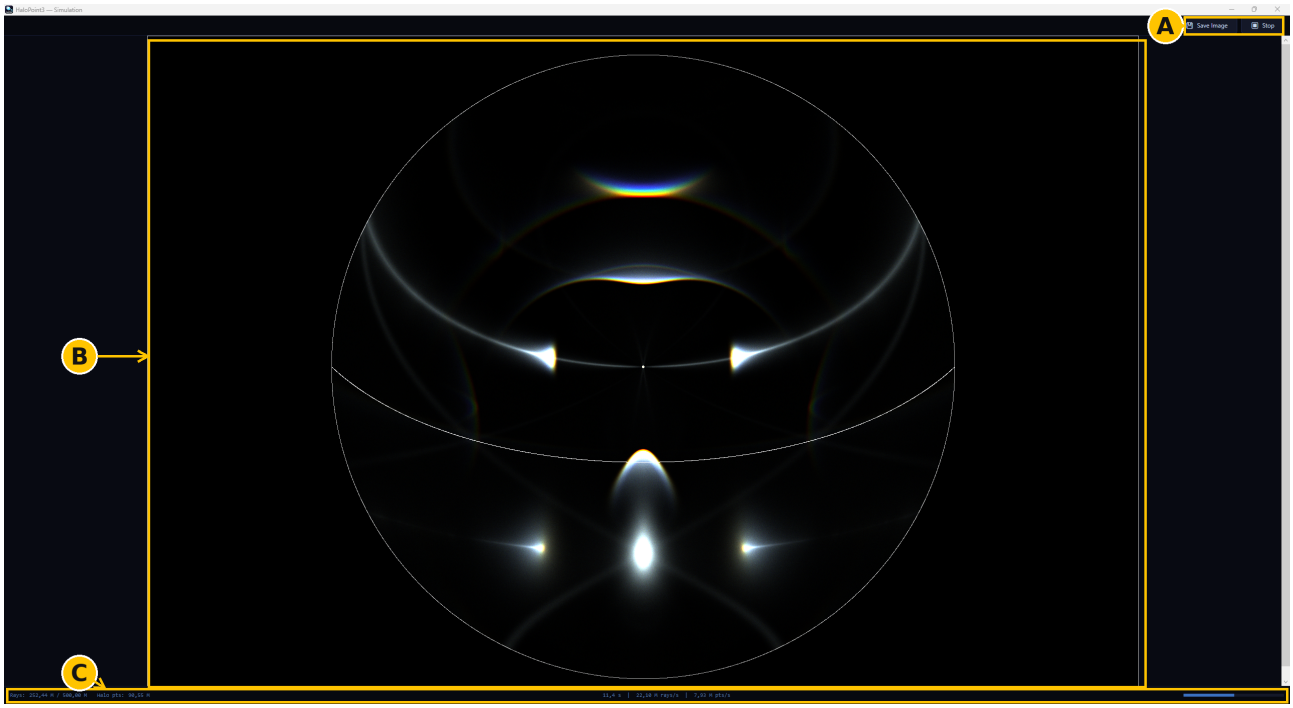


Figure 2. *A running simulation window. The top strip (**A**) carries the Save Image and Stop buttons. The scrollable canvas (**B**) shows the halo image accumulating at native resolution over the scene overlays. The bottom strip (**C**) reports live statistics: rays traced of the total, halo points plotted, elapsed time with mean tracing rates, and a progress bar.*

elapsed time with mean tracing rates in rays/s and halo points/s, averaged over the whole run so they stay steady; and a progress bar that fills as the ray target is approached.

The canvas area. The image is shown at native size, one simulation pixel to one screen pixel; if it exceeds the display, the canvas scrolls in both directions so any region can be inspected at full resolution during the run. The horizon line, coordinate grid, and halo ring overlays drawn before the run remain visible in the background throughout, with halo dots accumulating on top.

Working with multiple simulation windows. Since each window is independent, you can change settings and launch further simulations while others are still running, making side-by-side comparison straightforward — the same setup at two sun elevations, say, or different shade levels. Closing one window does not affect the others.

2.9 Recording mode

Recording mode captures the full ray path data behind every halo dot produced during a simulation, storing it in a format that can later be examined in the Analysis Window. In a standard simulation, only the final dot positions are accumulated into the image — the details of how each ray traveled through its crystal are discarded. Recording mode preserves those details, enabling post-simulation analysis of which crystal encounters produce which halos and how the light traveled to get there. The Analysis Window, described in Chapter 5, is built around this recorded data.

Recording mode is configured in the SIMULATION ANALYSIS card on the Advanced Functions tab.

Controls. ENABLE RECORDING (off by default) activates the mode; a note reminds you that only dots landing within the camera's field of view are recorded. When the ray count in the General Parameters card exceeds 10 million, a warning panel appears: recording stores a record for every dot, so large counts generate substantial data and slow the run. For analysis rather than image production, a far smaller ray count is usually enough.

OUTPUT FILE, with its BROWSE... button, sets the destination for a `.hprb` file. Left blank, recording data is held in RAM — available to the Analysis Window immediately after the run but lost when the application closes or a new simulation starts. Giving a path writes the data to disk as the run proceeds, preserving it across sessions for later reloading or sharing.

OPEN ANALYSIS WINDOW AFTER SIMULATION opens the window automatically when the run finishes or is stopped, loading the `.hprb` file if a path was given or the RAM data otherwise. OPEN ANALYSIS WINDOW is a button, always active, that opens the window and prompts for any existing `.hprb` file — the entry point for revisiting an earlier recording.

Performance and engine behavior. Recording tracks ray path data alongside dot accumulation and is somewhat slower than a standard run. Combined with multiple scattering it falls back to the CPU regardless of the GPU setting, shown by an amber "Recording → CPU (MS)" badge; the CPU is markedly slower, so a modest ray count matters most in this case. Without

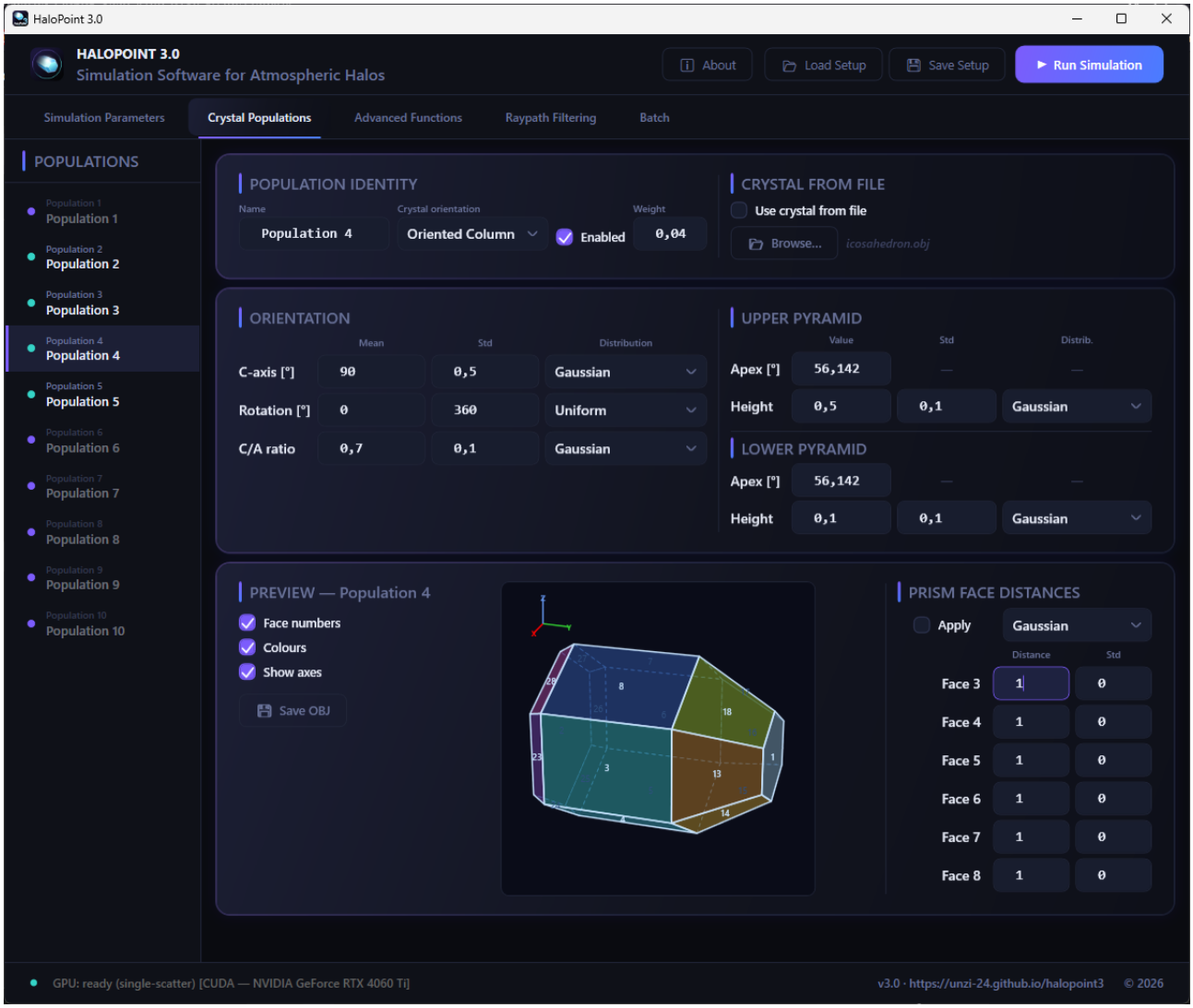


Figure 3. *The Crystal Populations tab. For each population a crystal and its orientation is defined, together with a weight that sets its relative abundance in the simulation. The* CRYSTAL FROM FILE *controls replace the parametric shape with a loaded OBJ mesh, which is classified as convex or non-convex on load. The ten crystal populations are accessed from the selector on the left.*

multiple scattering, recording uses the GPU and is much faster, though still slower than a non-recording GPU run. A cyan "Recording" badge confirms the mode is active.

3. Crystal Populations

3.1 Parametric crystals – dimensions

The parametric crystal model describes a hexagonal ice prism with optional pyramidal ends at the top and bottom. This covers the great majority of crystal forms responsible for known halos. The shape is defined by a small number of intuitive parameters, and for each ray an individual crystal is defined by sampling those parameters from a statistical distribution, so the population naturally represents a realistic spread of sizes and proportions rather than a single idealized shape.

HaloPoint 3.0 has three choices for a distribution function: Gaussian, uniform, and Laplacian[1]. The Laplacian distribution is available only for the c-axis tilt and the rotation angle; the other parameters offer Gaussian and uniform. Standard deviation is well defined for the Gaussian distribution. For the uniform distribution the standard deviation means the starting- and endpoints of the interval in which the parameter is allowed to vary in uniform manner. For the Laplacian distribution the entered value is not the standard deviation but the scale parameter *b*, which sets the width of the distribution about its mean. The distribution falls off exponentially on both sides of the mean, so it is more sharply peaked at the centre and has heavier tails than a Gaussian. The standard deviation of the

[1] Jiajie Zhang pointed out, that real ice clouds contain crystals spanning a range of sizes, each having their own std. When tilt is integrated over the full size distribution, the result follows a Laplace (double-exponential) distribution. See: https://github.com/LoveDaisy/ice_halo_sim/blob/main/doc/crystal-orientation-sampling.md

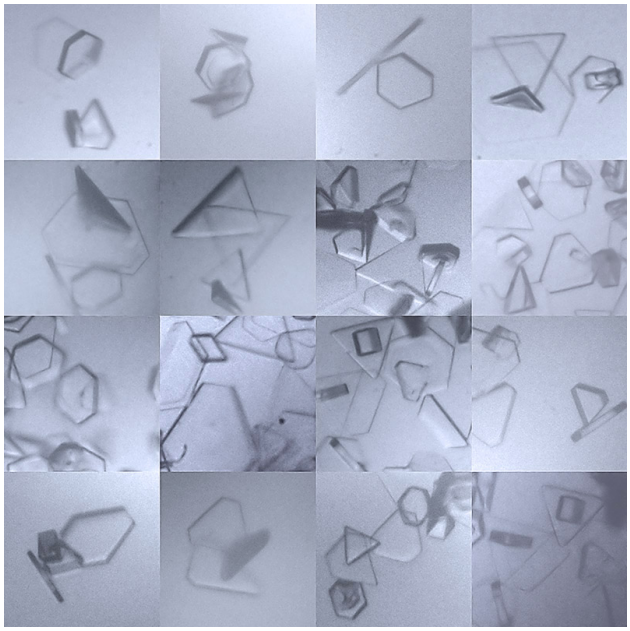


Figure 4. *Ice crystal profiles typically have various shapes, and symmetrical hexagons represent a small minority.*

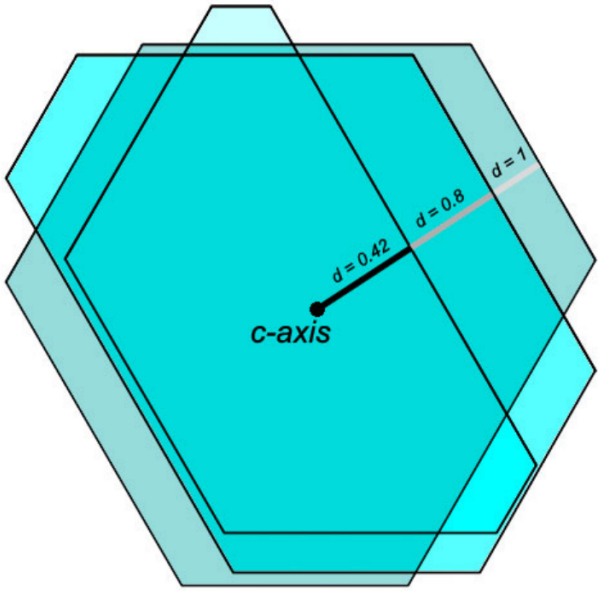


Figure 5. *By changing the distance of a face from the c-axis the profile of the crystal can be tuned.*

Laplacian is $b\sqrt{2}$, so a Laplacian and a Gaussian entered with the same field value do not have the same spread.

The controls described in this chapter are found on the CRYSTAL POPULATIONS tab (see Fig. 3), the second of the five main tabs.

Aspect ratio. C/A RATIO sets the ratio of the crystal's height along its c-axis to its a-axis width. An accompanying standard deviation field controls how much the ratio varies from crystal to crystal within the population, drawn from the distribution selected in the adjacent dropdown (Gaussian by default). Setting the standard deviation to zero gives every crystal in the population the same exact ratio. As the profile of the cross-section of the prism may vary, the a-axis width is as in the case of a regular hexagon.

Pyramidal caps. Each end of the prism can be given a pyramidal cap independently. The upper and lower (or "left" and "right") pyramids are configured separately but with identical controls.

APEX ANGLE sets the angle of the opposing faces of the pyramid, expressed in degrees. The default of 56.142° corresponds to the crystallographically most common apex angle of pyramid faces of ice crystals.

HEIGHT sets the height of the pyramid as a fraction of the maximum possible height — from 0 (no pyramid; the prism ends in a flat basal face) to 1 (a fully developed pyramid that comes to a complete point). At 0 no pyramid is present and the corresponding apex angle has no effect. Intermediate values produce truncated pyramids whose basal face is still visible. A mean and standard deviation are available for both upper and lower pyramid heights, with a distribution type selector.

Prism face distances. Ice crystal samples under microscope seldom contain regular hexagons (see Fig. 4). Instead, most of the crystals deviate from symmetrical shape, or are triangular or tabular in their cross-section. Therefore, in order to approach reality in simulations, it may be necessary to adjust the face distances from the c-axis (see Fig. 5).The six prism faces of a hexagonal crystal are conventionally numbered 3 through 8. In a regular hexagonal prism all six faces are equidistant from the crystal axis, meaning each face contributes equally to the cross-section. The PRISM FACE DISTANCES section allows the distance of each face from the c-axis to be adjusted independently.

This control is inactive by default — the APPLY FACE DISTANCES checkbox must be checked to enable it. When inactive, all six faces are treated as equally distant regardless of the values in the fields. When active, six distance fields labeled by face number become editable, each accepting a relative distance value. A standard deviation and distribution type control how much each face distance varies between individual crystals in the population.

3.2 Crystal orientation

Crystal orientation determines how the ice crystals in a population are aligned in the sky. Orientation is an important factor in deciding which halos can appear. Getting the crystal orientations right for a given halo display is usually one of the central tasks when matching a simulation to a photograph.

The c-axis of a hexagonal ice crystal is the crystallographic axis perpendicular to the basal plane and coincident with the crystal's sixfold symmetry axis. The a-axis is any crystallographic axis lying in the basal plane of a hexagonal ice crystal, perpendicular to the

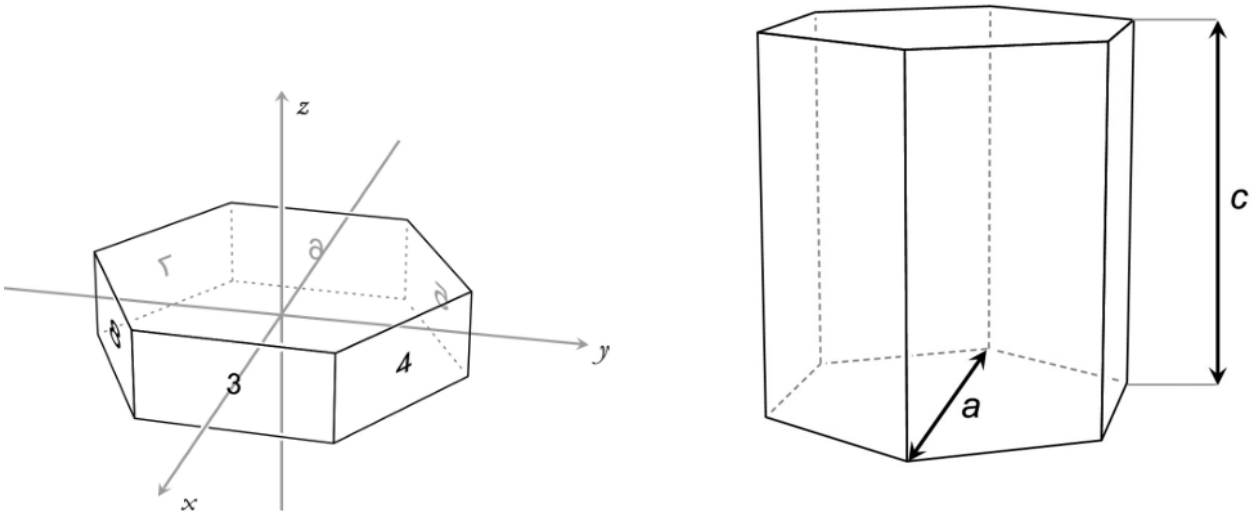


Figure 6. *On the left the ice crystal in its original orientation: c-axis is vertical and face 3 normal towards the positive x-axis and parallel to it. On the right the definition of aspect ratio is illustrated: it is the ratio of the length of the prism in c-axis direction to the width of a regular hexagon in a-axis direction (c/a).*

c-axis and parallel to two opposing prism faces. The orientation of a crystal in space is fully described by two angles: the tilt of the c-axis from the vertical, and the rotation of the crystal around its c-axis. The starting point of a hexagonal ice crystal in HaloPoint 3.0 is illustrated in Fig 6.

C-AXIS sets the mean tilt of the c-axis from vertical, with 0° pointing straight up and 90° horizontal. ROTATION sets the mean rotation about the c-axis. Each has an accompanying standard deviation and distribution type. C/A RATIO appears at the bottom of the same panel and controls the aspect ratio (section 3.1); it is grouped here because it interacts closely with orientation in determining the resulting halos.

A note on rotation. When a crystal rotates freely — as in the random, plate, and oriented column orientations — the rotation mean is insignificant, but the standard deviation must be 360° and the distribution uniform. When rotation is restricted, the direction of the face 3 normal matters for correct raypath labelling; for Parry and Lowitz orientations it is advisable to set the rotation mean to 90°, which turns the crystal into an orientation compliant with the widely used raypath conventions.

Orientation presets. The CRYSTAL ORIENTATION dropdown in the POPULATION IDENTITY card provides named presets that set the c-axis and rotation fields in one step. Selecting a preset overwrites the current values; any subsequent manual edit switches the dropdown to *User Defined*, which leaves all fields untouched for full manual control. Representative parameter values for the orientations included in the dropdown are summarized in Table 1.

Table 1. *Example ice crystal population parameters (in degrees) for the most-used orientation types. The values are arbitrary, given only to illustrate how each orientation is set up; fine tuning is necessary for each individual case.*

		Plate	Column	Parry	Random	Lowitz
c-axis	mean	0	90	90	0	0
	std	2	1	0.5	360	40
	distr.	G	G	G	U	G
rot.	mean	0	0	90	0	90
	std	360	360	0.5	360	0.5
	distr.	U	U	G	U	G

G = Gaussian, U = Uniform. Lowitz shown for the plate-like case.

3.3 OBJ file crystals (convex and non-convex)

Besides the parametric hexagonal prism, HaloPoint 3.0 can trace crystals of arbitrary shape supplied as OBJ files, allowing geometries that no parameterized prism can express — exotic polyhedra, truncated forms, or experimental research shapes. Any 3D modeling application that exports OBJ can produce them (or pencil and paper with perhaps a little aid from Matlab).

Loading and the convexity check. The CRYSTAL FROM FILE section is in the first card of the Crystal Populations tab; tick USE CRYSTAL FROM FILE and pick a file with BROWSE. . .. This replaces the parametric shape controls with the file geometry while leaving the orientation controls active. On load, the crystal is classified in the background and a badge reports the result: *convex*, *non-convex*, or *load error*. The distinction matters, because the two classes are traced by different engines with different capabilities.

Convex OBJ crystals. A convex crystal behaves exactly like a parametric one: it runs on the GPU, supports multiple scattering and recording, and has no face-count limit.

Non-convex OBJ crystals. Non-convex crystals are traced by a separate recursive engine, on both CPU and GPU, that follows rays which exit and re-enter the same body. Recording behaves differently on the two engines. On the CPU, a non-convex crystal is recorded in full — every internal surface encounter is captured, so the complete ray path is available for analysis. On the GPU, the kernel cannot expose the per-bounce face sequence of a non-convex crystal, so its recording is reduced to a stub: the chain notes that a non-convex crystal was traversed but not the individual bounces. The same stub applies to a non-convex crystal appearing as a *secondary* in a multiple-scattering chain, on either engine. In every case the halo image itself is unaffected — the limitation is only in the recorded ray-path detail. So for full non-convex ray-path analysis, run the recording on the CPU. Three parameters in the Processing Parameters card (section 2.5) bound the recursion: MAX RECURSION DEPTH (re-entries per ray; zero disables re-entry), RE-ENTRY INTENSITY CUTOFF (drops faint branches), and MAX DOTS PER RAY (NON-CONVEX) (guards degenerate geometries). The defaults suit most meshes.

What geometry is admissible. The non-convex tracer does not accept every mesh that is merely "not convex." It models a crystal as a union of *convex sub-parts joined face to face*, and the geometry must respect three conditions:

- **Built from convex pieces.** The body must be expressible as convex sub-parts that meet at shared, coplanar mating faces. Where two pieces join, both contribute a face on the same plane with opposite-facing normals; HaloPoint detects these pairs at load time and marks them as *seams*, passing rays through them without refraction or reflection. A re-entrant outline — a step, a notch, a stacked column — is fine as long as it decomposes this way.
- **No interpenetration.** Sub-parts may touch at seams but must not overlap or pass through one another. Interpenetrating solids produce interior faces the tracer cannot resolve, and rays reaching them are discarded.
- **No enclosed cavities.** Internal air pockets — bubbles or hollow voids inside the ice — are not supported. A ray that strikes a face from the ice side with no seam mate (the signature of an enclosed cavity) is silently dropped rather than refracted.

The practical consequence is a watertight, well-formed mesh of solid ice assembled from convex blocks. When a ray encounters a surface the engine cannot classify as either a true exterior boundary or a seam — because the mesh interpenetrates, contains a cavity, or is otherwise malformed — that ray is discarded rather than contributing a spurious dot. This keeps the halo image physically correct, but means a flawed mesh quietly loses rays instead of raising an error. If a non-convex crystal produces a dimmer or sparser result than expected, the geometry is the first thing to check: confirm it is a clean union of non-overlapping convex parts with no trapped interior space.

The seam tolerance. How strictly mating faces must align to register as a seam is set by COPLANAR SEAM EPSILON in the Processing Parameters card — the angular tolerance within which two face normals must be antiparallel and coincident. The default of 0.001 suits meshes from standard modeling tools. Raise it only if genuine seams are being missed through numerical imprecision; lower it only if legitimate near-coplanar exterior faces are being wrongly suppressed.

3.4 Managing multiple populations

HaloPoint 3.0 supports up to ten crystal populations running simultaneously in a single simulation. This makes it possible to model a realistic halo display in which different crystal populations coexist in the same cloud, each with its own shape, orientation, and relative abundance.

The population list. The left side of the Crystal Populations tab shows all ten populations as a vertical list. Each entry displays a small colored dot, the population number, and its name below it. A cyan dot indicates the population is active and will contribute halo dots to the simulation; a dim dot indicates the population is inactive and will be ignored.

Enabling and naming populations. Each population is enabled or disabled using the ENABLED checkbox in the first card of the right-hand panel. Populations can be given descriptive names in the NAME field — for example "Plates" or "Columns" — which appear in the list and are also used to identify populations in the Raypath Filtering tab and the Analysis Window.

Weight. The WEIGHT field controls the relative proportion of rays directed at each population during a simulation. Weights do not need to sum to any particular value — only the ratios matter. If two populations each have a weight of 1.0, each receives half the total rays. Setting a population's weight to zero is equivalent to disabling it, though the ENABLED checkbox is the cleaner way to exclude a population entirely.

3.5 Crystal preview

The crystal preview is a 320×320 pixel interactive 3D rendering of the current population's crystal geometry (see Fig 7). It sits in the left portion of the third card and updates automatically whenever the crystal shape, aspect ratio, pyramids, or face distances are changed.

It gives immediate visual feedback that the configured geometry and orientation matches the intended crystal form before committing to a full simulation run. The crystal is oriented in the preview to reflect the mean c-axis tilt and rotation values set in the orientation controls.

The view angle can be rotated freely by clicking and dragging anywhere in the preview canvas. Dragging horizontally rotates the crystal around the vertical axis; dragging vertically tilts it. This makes it straightforward to inspect any face of the crystal from any angle.

The SAVE OBJ button exports the current population's mean crystal geometry as an OBJ file. The exported file contains all vertices, per-face normals, and triangulated face definitions, and can be opened in any 3D modelling application for external inspection or verification of the configured geometry.

Three display options appear next to the preview canvas as checkboxes.

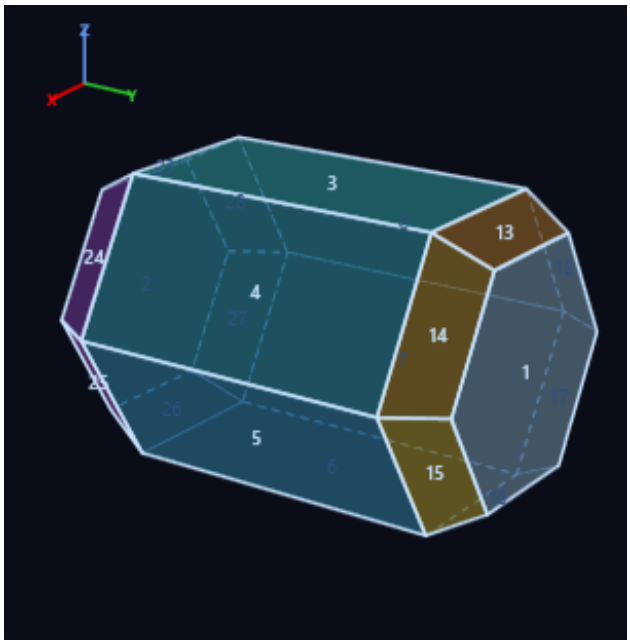


Figure 7. *A preview of the parametric ice crystal is shown on Tab 2.*

SHOW FACE NUMBERS overlays the face index of each visible face at its centroid. These numbers correspond directly to the face indices used in ray path descriptions in the Raypath Filtering tab and the Analysis Window, making it straightforward to identify which face a filter rule or a recorded ray path is referring to. For OBJ crystals face numbers are assigned at load time strictly by the order in which face (`f`) lines appear in the OBJ file: the first face is 1, the second 2, and so on. The numbering is positional only and has no relation to anything inside the OBJ file itself, since the format has no concept of a face index — only vertex indices. Faces split into multiple triangles during tessellation all keep the index of their original face.

SHOW COLOURS assigns a distinct color to each face group: basal faces are rendered in grey-blue, prism faces in cyan, upper pyramid faces in amber, and lower pyramid faces in violet.

SHOW AXES draws a small X/Y/Z axis indicator in the upper-left corner of the preview canvas, showing the orientation of the crystal's coordinate frame as the view is rotated.

4. Advanced Functions

4.1 Raypath filtering

Raypath filtering allows specific halo forms to be isolated or highlighted within a simulation based on the exact path that light took through the crystal. Rather than showing every halo dot regardless of origin, a filter defines rules about which ray paths are considered matching; dots that match are rendered normally or in a highlight color, while non-matching dots can be hidden, dimmed, or left unchanged.

Enabling the filter. The Raypath Filtering tab contains a master ENABLE FILTERING checkbox at the top. When checked, a small indicator dot appears on the tab label to confirm the filter is armed. The filter is applied from the very first dot of the simulation and cannot be changed or toggled while a simulation is running. To change the filter, stop the simulation, adjust the rules, and run again.

Population scope. Filtering is applied per population. The top section of the tab shows a checklist of currently active populations; only those checked here are subject to the filter rules. Populations not selected in this list are treated as unfiltered and rendered according to the UNFILTERED DOTS setting described below.

Unfiltered dot behavior. Three options control how dots that do not match the filter are displayed. *Hide* suppresses them entirely, leaving only matching dots visible. *Dim* renders them at a reduced intensity — the fraction of normal brightness is set in an accompanying text field, defaulting to 0.30 — which preserves spatial context while keeping dots that passed the filter visually prominent. *Normal* renders all dots at full intensity regardless of filter status, probably not very useful mode in most cases.

Filtered dot color. By default, matching dots are rendered in their normal spectral color. Selecting OVERRIDE COLOR replaces the spectral color of matching dots with a fixed user-defined color, entered as a six-digit hexadecimal RGB value. A color swatch and a set of preset color chips are provided for quick selection.

Rules. Each selected population has its own set of filter rules listed beneath the POPULATIONS TO FILTER section. Rules within a population are combined with AND logic — a dot must pass every rule in the set to be considered matching. Multiple alternative face sequences within a single sequence rule are combined with OR logic.

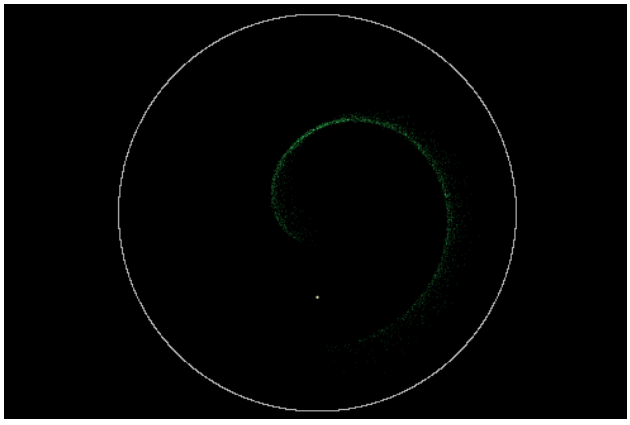


Figure 8. *Example of filtering: Exact raypath 3-1-5 (Wegener) without C6-symmetry applied.*

Eight rule types are available, added via a dropdown labeled ADD RULE.

Entry face passes dots whose ray entered the crystal through one of a specified set of face indices. *Exit face* passes dots whose ray exited through one of a specified set. *Nth hit face* passes dots where the ray's Nth surface encounter was on a specified face. If user wants to filter using more than one face, the face sets are entered as comma-separated integers using the standard face numbering.

Encounter count passes dots where the total number of surface encounters with the crystal equals a specified value. A ray that enters through a prism face, reflects internally once, and exits through another prism face has an encounter count of 3. This filter rule only accepts one value.

Exact raypath passes dots whose full sequence of face encounters matches one of the specified sequences, written in hyphen-separated notation — for example, 3-5 for a ray entering face 3 and exiting face 5, or 3-1-5 for a three-face path. Multiple alternative sequences are separated by commas. A C6 SYMMETRY checkbox expands each sequence to include all six rotationally equivalent variants automatically, which is almost always desirable for hexagonal crystals. Exact raypath filtering can handle paths of up to eight surface encounters. Longer paths are traced and contribute to the simulation normally, but fall outside the reach of an exact raypath rule.

Deflection angle passes dots within a specified angular range from the sun, in degrees.

Filtering with multiple scattering. In HaloPoint 3.0 only one filter rule remains available when multiple scattering is enabled:

- The *Scatter level* rule selects dots by the number of crystals the ray passed through — 1 for a single crystal, 2 for two, 3 for three.

All other rule types — entry face, exit face, Nth hit face, encounter count, exact face sequence, and deflection angle — describe the geometry of one specific crystal pass, which is not tracked through a multi-crystal chain. These rules are shown grayed out while multiple scattering is enabled and take no part in the filter. To use them, disable multiple scattering.

4.2 Batch processing

Batch processing allows a series of simulations to be queued and run sequentially without further interaction. Each simulation in the batch uses a different value of one or more parameters, sweeping through a specified range from a start value to an end value in user defined fixed steps. All other settings — crystal populations, camera, wavelength, GPU configuration — remain unchanged across the batch. Output files are saved automatically to a folder of your choice as each simulation completes.

Enabling batch mode. The Batch Processing tab contains an ENABLE BATCH checkbox at the top. When checked, the batch configuration becomes active and the Run Simulation button in the main window launches the entire batch sequence rather than a single simulation. A summary line below the configuration describes the number of steps and output destination, updating live as settings are changed.

Single-parameter mode. In single-parameter mode, one parameter is swept across a range. A dropdown selects the parameter — either a global simulation parameter or a per-population crystal parameter — and three fields define the sweep: START, END, and STEP. The number of resulting simulations is calculated automatically and displayed as a step count badge.

The global parameters available for sweeping are sun elevation, sun diameter, wavelength minimum and maximum, number of rays, shade levels, focal length, and multiple scattering probability. The per-population parameters are c-axis tilt mean and standard deviation, rotation mean and standard deviation, C/A ratio mean, upper and lower pyramid height mean, and each of the six prism face distances. Population parameters require selecting which population to apply the sweep to from an accompanying dropdown.

Multi-parameter mode. Multi-parameter mode sweeps several parameters at once, running every possible combination of their values. Each parameter is entered as a row in a table with its own start, end, step, and scope settings. For example, sweeping sun elevation across three values and c-axis tilt standard deviation across four values produces twelve simulations — one for each pairing of the two.

In multi-parameter mode, the output folder structure can be set to flat — all files in a single folder, with parameter values encoded in each filename — or to sub-folders by first parameter, which creates one sub-folder per value of the first parameter sweep and places the remaining parameter combinations inside each sub-folder.

Output files. Each simulation step automatically saves its result to the configured output folder. The

output filename is built from a user-defined prefix followed by the step number, the parameter value, and a tag identifying the file format — for example `batch_01_val10.0_png8.png`. Five image formats and one ray path recording can be enabled independently, and any combination can be written for the same run:

- **PNG, 8-bit** (enabled by default) and **PNG, 16-bit** — the tone-mapped display image, at 8 or 16 bits per channel.
- **TIFF, 8-bit** and **TIFF, 16-bit** — the same tone-mapped image in a TIFF container.
- **TIFF, 32-bit float** — the raw high-dynamic-range accumulator (described in subsection 2.8 Simulation window).
- **Ray path recording** (`.hprb`) — the full ray path data for later analysis in the Analysis Window.

Each enabled format is written with its own filename tag (`_png8`, `_png16`, `_tiff8`, `_tiff16`, `_tiff32f`), so the files never collide when several are saved together.

A WRITE BATCH_README.TXT checkbox (enabled by default) writes a plain-text summary file to the output folder recording all simulation parameters, which values were swept, and the start and end times of the batch run. This makes it straightforward to reconstruct what each output file represents when returning to a set of results later.

Running and cancelling a batch. Clicking Run Simulation with batch mode enabled launches the sequence. Each simulation opens a simulation window, runs to completion, saves its output files, and closes automatically before the next step begins. The Run button in the main window changes to show the current step number out of the total, and can be clicked again during a batch run to cancel the remaining steps. Any simulation currently in progress when the cancel is requested is stopped immediately; steps that have already completed and saved their output are not affected.

5. Analysis Window

5.1 Opening the analysis window

The Analysis Window is where ray path data from a recording simulation can be explored in detail. It presents the simulation image alongside tools for selecting regions of interest and examining the ray paths that produced the halo dots within them. The window is independent of the main window — it opens maximized as its own entry in the taskbar and can remain open alongside any number of simulation windows.

How to open it. There are two ways to reach the Analysis Window. The most direct is to check OPEN ANALYSIS WINDOW AFTER SIMULATION in the Simulation analysis card on the Advanced Functions tab before running a simulation; when the simulation ends or is stopped, the Analysis Window opens automatically and loads the recorded data without any further action from the user.

The second way is to click the OPEN ANALYSIS WINDOW button in the Simulation analysis card at any time and open a file dialog for selecting an existing `.hprb` file from disk. This is also the entry point for revisiting recordings from previous sessions.

What the window shows. When the window opens it displays the simulation image produced by the recording run, rendered from the ray path data using the same camera projection and scene settings that were active at the time. The image is shown at full size in a scrollable canvas; for large output images, scrollbars appear to allow navigation. The raypath filter that was active during the recording run is also applied to the analysis image, so the two views are consistent.

Four selection tools are available in the toolbar area of the window. The rectangle tool, active by default, lets you drag a rectangular region directly on the simulation image; all halo dots within the rectangle are selected. The lasso tool works similarly but accepts a free-form drawn boundary, useful for isolating an irregularly shaped halo arc or a small cluster of dots. The coordinate tool bypasses drawing entirely and instead accepts explicit azimuth and elevation ranges as numeric values, selecting all dots that fall within the defined sky region. A fourth option, SELECT ALL, includes every recorded dot in the dataset. Once a selection is made, a count of the selected dots is shown in the status bar, and the INSPECT SELECTION button opens the Ray Path Report window described in sections 5.2 and 5.3.

The status bar at the bottom of the window reports the source of the loaded data — either a filename for file-backed recordings or “In-memory recording” for data kept in RAM — along with the total number of recorded ray paths available for analysis. As the mouse moves over the projection, the status bar reports the sky coordinates beneath the cursor: azimuth and elevation in degrees, together with the angular distance from the sun. Azimuth is displayed in the range -180° to $+180^\circ$, with the sun at 0°. Additionally, a measurement stick can be launched from the toolbar.

5.2 Raypath histogram and bins

Clicking INSPECT SELECTION in the Analysis Window opens the Ray Path Report window (see Fig. 9). This window analyses all the ray paths within the selected region and presents the results as a histogram on the left side of the window, with a ray path illustration on the right. This section covers the histogram; the illustration is described in section 5.3.

What the histogram shows. The histogram lists every distinct ray path found among the selected halo dots, sorted by frequency from most to least common.

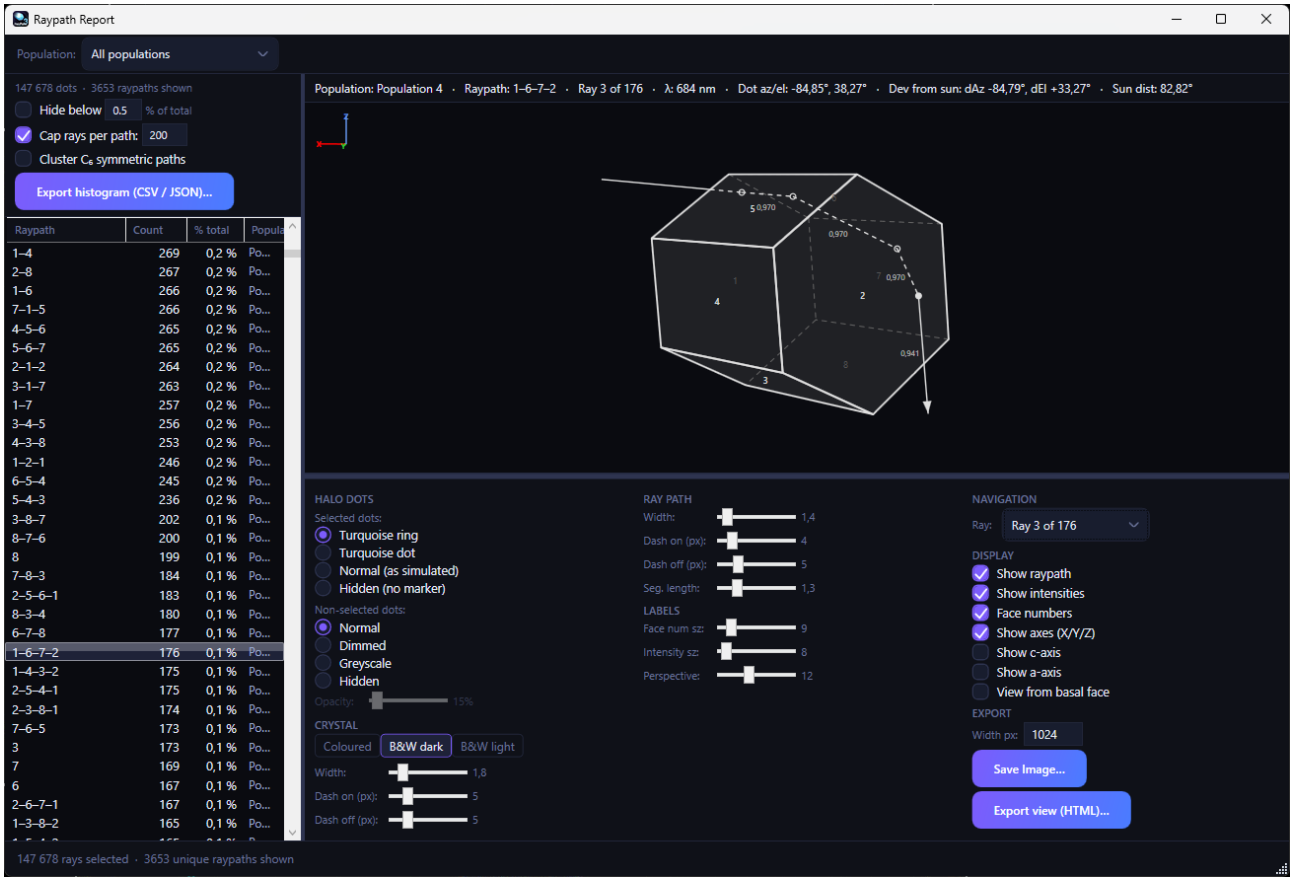


Figure 9. *The Raypath Report window, listing the ray paths that contributed to the simulated halo together with their relative frequencies. The crystal and ray path image can be exported as PNG image and raypath data can as CSV, JSON, or a standalone HTML file.*

Each row represents one unique path — identified by the sequence of crystal faces the ray encountered, written in hyphen-separated notation such as 3–5 for a two-face path or 3–1–5 for a three-face path, where the numbers are the standard face indices for parametric crystals.[2] Alongside the face sequence, each row shows the number of recorded dots that followed that path and their percentage of the total selection.

When the recording contains dots from more than one crystal population, a dropdown at the top of the window lets you choose which population to examine. Selecting *All populations* adds a population column to the histogram, so each row identifies both the ray path and which population it originated from — useful for seeing how different crystal types contribute to the same region of the sky. When a specific population is selected instead, the histogram shows only the paths from that population, and an extra column appears showing each path's percentage of the full combined selection across all populations. This allows you to read both the within-population share and the overall share at the same time.

Clicking any row in the histogram selects that path as the active bin. The dots in the Analysis Window image that correspond to the selected path are immediately highlighted in the simulation image — by default they are shown with a turquoise ring marker — while all other dots are rendered at reduced opacity or in their normal state depending on the display settings. This direct link between the histogram and the image makes it easy to locate exactly where in the sky a given ray path contributes dots. Multiple paths can be selected at once by holding CTRL or SHIFT while clicking rows, in which case the highlighted dots are the union of all selected paths, while the crystal illustration and RAY dropdown continue to follow the most recently clicked row.

Histogram controls. Three controls above the histogram list adjust which paths are shown and how the rows are populated.

HIDE BELOW is a checkbox and percentage field that suppresses rows whose dot count falls below the specified fraction of the total. The default of 0.5% filters out rare paths that would otherwise create long tails of marginally populated rows. Unchecking the box shows all paths.

CAP RAYS PER PATH limits the number of individual ray records shown per histogram row. When a path appears thousands of times in the recording, only the first *N* records are loaded into the row for display purposes — the full count is still reported accurately, but

[2] For crystals defined in OBJ files, the face numbers are assigned during file parsing, starting at 1 and incrementing by 1 for each valid `f` line encountered. The numbering is strictly positional — it reflects the order `f` lines appear in the OBJ file.

only *N* examples are available as turquoise markers on the Analysis Window and for the ray path illustration on the right. The default cap of 200 is generous enough for illustration purposes and keeps the interface responsive. The cap can be disabled or raised if needed.

CLUSTER C_6 SYMMETRIC PATHS groups rotationally equivalent ray paths together into single histogram rows. A hexagonal crystal has sixfold rotational symmetry around its c-axis, meaning that path 3–5 and path 4–6 and path 5–7 and so on are physically identical up to rotation. With clustering enabled, all six rotationally equivalent variants of a path are counted together and shown as a single row with a canonical label. This is most meaningful for populations with free rotation (rotation standard deviation of 360°) and is disabled automatically for populations where the rotation is constrained. Clustering reduces clutter significantly in the histogram when a large selection spans multiple equivalent prism orientations. There are also other symmetries and logical ways that could be applied to sort the raypaths, but these are not implemented. The raypath export feature (see "Exporting the histogram" in 5.3) allows the user to write own scripts for arranging and sorting raypaths outside of HaloPoint 3.0.

5.3 Raypath illustration

The right side of the Ray Path Report window shows a three-dimensional illustration of the crystal and the ray path through it for the histogram bin currently selected on the left. This illustration is drawn from the actual recorded geometry of a specific ray — the exact crystal shape, orientation, and face sequence.

What the illustration shows. The crystal is rendered in perspective using the same face coloring and edge style as the crystal preview in the Crystal Populations tab. The ray path is superimposed on it: the incoming sun ray is drawn as a line segment arriving at the entry face, each internal surface encounter is marked with a small dot on the face where it occurred, and the outgoing ray exits through the final face with an arrowhead indicating the direction of travel. For multiple-scattering ray paths, all crystals in the chain are shown side by side, with the ray connecting them.

Adjusting the view. The illustration is fully interactive, and what you see on screen is exactly what gets exported — both the PNG and the HTML export (described below) capture the view as it currently appears. Drag with the mouse on the canvas to rotate the crystal and ray path freely in three dimensions; the PERSPECTIVE slider controls the strength of the perspective projection, from near-orthographic to strongly foreshortened. The canvas itself can be reshaped to whatever aspect ratio suits the geometry: drag the vertical splitter between the histogram and the illustration to change its width, the horizontal splitter beneath the canvas to change its height, or resize the whole window. A tall ray path reads better in a portrait canvas, a wide multiple-scattering chain in a landscape one. Arranging the rotation, perspective, and canvas shape to your liking before exporting is the intended workflow — there are no separate export-framing controls because the export simply reproduces the current view.

Navigating between rays. Each histogram bin may contain dozens or hundreds of individually recorded rays that all share the same face sequence. The RAY dropdown below the canvas steps through these examples one by one. Switching between rays updates the illustration with the actual geometry of that specific recorded instance — a different crystal shape, a different orientation, a slightly different entry point — allowing the user to see the natural variation within the bin. The dropdown also updates the information strip above the canvas, showing the population name, the ray path label, and the wavelength of the selected ray.

Display options. Several checkboxes control what additional information is overlaid on the illustration. SHOW RAY PATH toggles the ray line and encounter dots on or off, leaving just the crystal. SHOW INTENSITIES adds small numerical labels at each ray segment showing the fraction of intensity that is left in the ray. SHOW FACE NUMBERS overlays the face index on each visible face, using the same numbering convention as the crystal preview. SHOW C-AXIS and SHOW A-AXIS draw the corresponding crystallographic axes for checking the orientation. VIEW FROM BASAL FACE rotates the view to look directly along the c-axis from above, giving a clear top-down view of the prism cross-section and the ray path through it — particularly useful for prism paths where the horizontal geometry matters most. The line widths, dash patterns, and label font sizes for both the crystal edges and the ray can be tuned in the control panel, and the appearance can be switched between the coloured scheme and black-on-white or white-on-black for figures.

Dot highlighting in the Analysis Window. The illustration and the Analysis Window image are linked. When a histogram bin is selected, the halo dots in the Analysis Window that correspond to that bin are highlighted, and all other dots are adjusted according to the display settings in the SELECTED DOTS and NON-SELECTED DOTS control groups. Selected dots can be shown as a turquoise ring marker (the default), a filled turquoise dot, in their normal simulated color, or hidden entirely. Non-selected dots can be shown normally, dimmed to a specified opacity, converted to greyscale, or hidden. These settings allow you to isolate the spatial distribution of a specific ray path in the sky image while retaining as much or as little context from the surrounding halo display as needed.

Exporting the crystal-and-ray-path illustration. The illustration can be saved as a PNG file using the SAVE IMAGE button at the bottom of the panel. The output width in pixels can be specified before saving; the height follows automatically from the current canvas

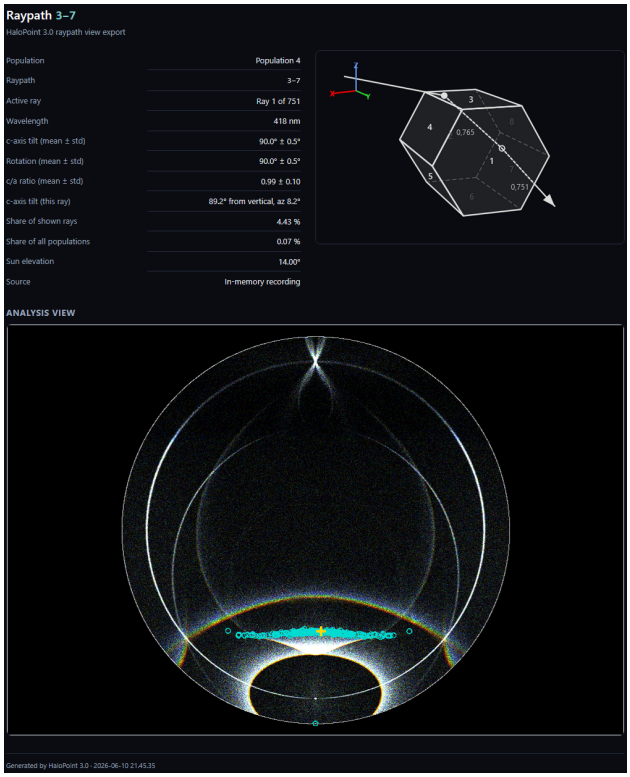


Figure 10. *The* EXPORT VIEW *button produces a single self-contained HTML file that captures the whole report for the active ray in one document.*

aspect ratio, and all line widths and font sizes scale to match, so the saved image reproduces the on-screen view at the chosen resolution. The default filename includes the ray path label of the active bin.

Exporting a complete view. The EXPORT VIEW button produces a single self-contained HTML file that captures the whole report for the active ray in one document. It combines the crystal-and-ray-path illustration, the corresponding Analysis Window sky view with its highlighted dots and the gold marker on the active ray, a metadata block, and the crystal's orientation and shape-distribution parameters (see Fig. 10). The sky view is re-rendered cleanly at the export resolution rather than scaled from the screen, so the halo dots stay crisp. As with the PNG export, the crystal-and-ray-path illustration is reproduced exactly as you have arranged it — rotation, perspective, and canvas shape included — so set up the view to your satisfaction first. The resulting file is fully portable: it embeds its images and needs no external files, and opens in any web browser.

Exporting the histogram. The histogram itself can be exported as data using the EXPORT HISTOGRAM button beneath the histogram controls, in either CSV or JSON format. The export lists every ray path in the current histogram with its dot count, its share of the shown selection, and — where applicable — its population and its share of the full multi-population selection. CSV suits a spreadsheet; JSON additionally records the recording's metadata (source, sun elevation, total ray counts, and clustering state) in a header alongside the per-path rows, making it convenient for scripted analysis.

Appendix — Simulation path performance and capabilities

HaloPoint 3.0 selects the appropriate simulation engine automatically based on the combination of features enabled. Different combinations route to different internal engines, each with its own set of capabilities and performance characteristics. Table 2 below summarises which features are available in each configuration, and Table 3 gives a feel for the throughput to expect on different hardware.

A few clarifications on entries that may be unexpected. When recording and multiple scattering are both enabled with GPU acceleration on, the simulation falls back to the CPU engine silently. This is because the GPU recording engine does not support the multiple-scattering kernel. The amber "Recording → CPU (MS)" notification confirms the fallback is active. The result is correct but slower. Multiple scattering combined with non-convex OBJ crystals also runs on the CPU engine, even when GPU acceleration is on. The fallback is automatic and produces correct results.

Non-convex OBJ crystals in GPU recording mode produce a simplified record: the full sequence of surface encounters inside the non-convex crystal is not captured per ray-face encounter, and the recorded chain instead notes only that a non-convex crystal was traversed at that point. The halo image is unaffected; the limitation applies only to the ray path detail available in the Analysis Window. The raypath filter is supported across all engine paths. On the CPU engines, filtering is applied dot by dot as rays are traced. On GPU engines, filter metadata is embedded in each halo dot before it is passed to the rendering pipeline. The CPU engine is used whenever GPU acceleration is disabled, regardless of other settings. On machines without a compatible GPU, this is the only available path and all features work correctly within it.

Suggestions for further reading

A few recommendations for those who want to explore halos further. Walter Tape's *Atmospheric Halos* (American Geophysical Union, 1994) and its successor, *Atmospheric Halos and the Search for Angle X* by Tape and Jarmo Moilanen (American Geophysical Union, 2006), are excellent treatments of the subject. Finnish readers also have Marko Riikonen's fine *Halot: Jääkidepilvien valoilmiöt* (Ursa, 2011). Finally, Skywarden observation service maintained by Ursa Astronomical Association at https://www.taivaanvahti.fi/ collects sightings and photographs from observers — a good place to see what these simulations are really trying to reproduce.

Table 2. *Simulation functional path matrix. Each cell states the support level for a combination of* crystal type *(column) and* execution mode *(row pair).*

Mode		Parametric	Convex OBJ	Non-convex OBJ
Single scattering	CPU	✓ Full	✓ Full	✓ Full
	GPU	✓ Full	✓ Full	✓ Full
Multiple scattering	CPU	✓ Full	✓ Full	✓ Full
	GPU	✓ Full	✓ Full	↩ CPU fallback
Recording (single scatter)	CPU	✓ Full	✓ Full	✓ Full
	GPU	✓ Full	✓ Full	△ Partial (primary)
Recording (multi-scatter)	CPU	✓ Full	✓ Full	△ Partial (secondary)
	GPU	↩ CPU fallback	↩ CPU fallback	↩ CPU fallback
Ray filter (live)	CPU	✓ Full	✓ Full	✓ Full
	GPU	✓ Full	✓ Full	✓ Full
Batch	CPU	✓ Full	✓ Full	✓ Full
	GPU	✓ Full	✓ Full	✓ Full

Legend: ✓ Full fully supported with each encounter data; △ Partial (primary) when a non-convex crystal is the primary scatterer, GPU recording still produces a correct halo image, but the `.hprb` file holds no encounter details for that crystal — only the fact that the ray passed through it; △ Partial (secondary) the CPU recording engine captures full face encounter details for a non-convex *primary* crystal. A non-convex crystal reached later in the chain, as a *secondary* or deeper multiple-scattering target, is logged only as a placeholder: its orientation and surface encounters are not recorded; ↩ CPU fallback GPU multiple scattering with non-convex crystals and GPU recording combined with multiple scattering is unsupported and silently falls back to the CPU recording engine (the amber "Recording → CPU" is shown on the notification bar) The CPU-fallback cells carry the same recording limitation as the CPU rows above: a non-convex crystal reached as a secondary or deeper scattering target is logged only by its presence in the chain, with its orientation and surface encounters omitted.

Table 3. *Approximate throughput in millions of rays per second (Mray/s) across available hardware. The NVIDIA column is GPU-accelerated; the three Intel columns are CPU-only. Where a crystal type was tested with two meshes, a plate crystal and an icosahedron (an unrealistic but high-face-count shape, chosen to stress the tracer), both are listed as* value *(*shape*). Figures are indicative and vary with driver version, settings, and scene complexity.*

Mode / crystal type	RTX 4060 Ti (GPU)	i7-14700F (CPU)	i5-1135G7 (CPU)	i7-13620H (CPU)
Single scattering				
Parametric	26	0.7	0.4	0.55
OBJ convex	29 (plate)	12 (plate)	3.6 (plate)	4.6 (plate)
	22 (icosa.)	4.5 (icosa.)	1.7 (icosa.)	4.0 (icosa.)
OBJ non-convex	1.3	2.5	0.9	1.8
Multiple scattering				
Parametric	4.2	0.2	0.1	0.06
OBJ convex	6.1 (plate)	4.6 (plate)	1.5 (plate)	1.5 (plate)
	4.4 (icosa.)	1.8 (icosa.)	0.5 (icosa.)	0.4 (icosa.)
OBJ non-convex	n/a	0.6	0.4	0.4
Ray-path recording (single scatter), 1M rays				
Parametric	1.1	0.35	—	—
OBJ convex	1.5 (plate)	1.5 (plate)	—	—
	0.6 (icosa.)	0.5 (icosa.)		
OBJ non-convex	n/a	0.9	—	—

Units are millions of rays per second. "—" indicates a combination that was not measured. For non-convex crystals, note that the CPU often matches or exceeds the GPU — the reason probably being the very complex structure of the non-convex GPU kernel. Perhaps in high-end NVIDIA hardware the GPU is stronger.